Vietnamese-German University

Hospital Assignment System

by

Nguyen Phuoc Thanh – 10422075

Nguyen Kim Minh Thanh – 18735

Nguyen Huu Trien – 17813

Project report submitted in partial fulfilment of the

requirements for a graded credit for the module

Compulsory Elective 1

Modelling of Concurrent Systems with PROMELA

in Winter Term 2025/26

Submission Date: 17th December 2025

Module Instructor: Prof. Dr.-Ing. Ruth Schorr

Vietnamese German University

Table of Contents

Chapter 1. Introduction

Hospital Assignment System is a Promela project that aims to model a simplified version of a working hospital. In this chapter, the setup of the hospital is introduced along with the requirements.

1.1 Problem Statement (Nguyen Phuoc Thanh – 10422075)

The hospital opens 8 hours a day, serves three different types of customers and has three departments with distinct setups. The customer types include:

- Regular: a normal customer, receiving standard service.
- Insured: have higher priority than regular customers when waiting to get treatment
- VIP: have higher priority than insured customers with additional privileges depending on the destination department.

There are three departments: A, B, and C.

In department A, each treatment lasts 10-20 minutes and requires a doctor and a machine. There are 3 doctors sharing 2 machines. In this department, VIP customers can reserve a machine the moment they enter the hospital.

In department B, there is a group of 2 junior doctors and 1 senior doctor working with each other. Every customer must be examined by a junior doctor for 3-5 minutes to check the seriousness of the case. If the case is mild, the junior doctor can provide treatment (10-15 minutes), otherwise the junior doctor will refer the case to the senior doctor. However, VIP customers are granted privilege to go directly to the senior doctor, taking 15-20 minutes per treatment.

In department C, there are 2 operating rooms, 1 cleaning team, and 1 pre-op room. After each surgery (20-30 minutes) the operating room will be dirty and need the cleaning team to clean it (5-10 minutes) before the next surgery. The Pre-Op Room, with a capacity for 1 customer, is a place the customer must enter and stay for 3-5 minutes to prepare for surgery. The benefit of VIP customers in this department is the ability to kick a non-VIP customer waiting in the Pre-Op Room out and take their spot.

The workflow of the project including five main steps, represented in Fig. 1. There are three lines of customers waiting outside of the hospital, the Gatekeeper takes the waiting customers and decides whether they can enter the hospital. Accepted

customers are routed to their respective departments, where they are provided with treatment in step 4. Treated customers will be reported in the final step.

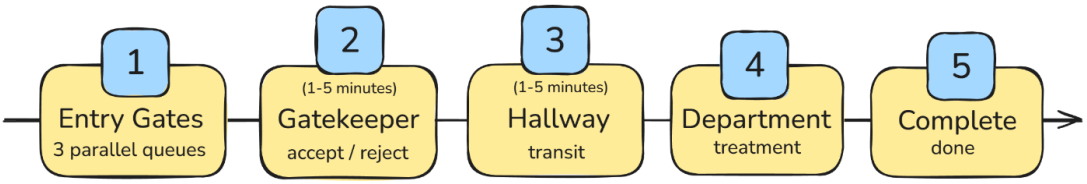


Fig. 1. Hospital workflow

1.2 Requirements (Nguyen Phuoc Thanh – 10422075)

The system needs to be implemented correctly as stated in 1.1, while avoiding concurrency problems: deadlock, live-lock, starvation, and mutual exclusion. In this project, starvation is solved by ensuring that every customer accepted by the Gatekeeper will be eventually treated while the hospital's working time is maintained around 8 hours. The other customers in step 1 in Fig. 1 that come late and cannot be treated in time will be rejected.

Chapter 2. Implementation

This section covers how the system is modelled in Promela, including implementations for Entry Gates, Gatekeeper, Hallway, Departments (Fig. 1), and a client-server methodology to synchronize time between these processes.

2.1 Time Synchronization (Nguyen Phuoc Thanh – 10422075)

The working time of the hospital is controlled by a time server. A global variable `globalTime`, in Fig. 2, represents how many minutes that has passed from the moment the hospital opened. This variable is increased one by one by `ClockTicking` process (Fig. 2) until it passes closure time (480 minutes) and all accepted customers in the day have been cured. In addition, to ensure the consistency of the time value `globalTime` in every process, before increment the `globalTime`, the `ClockTicking` server sends notifications (TICK messages) to the client processes via `timeNotify` channels (Fig. 2) and waits for their acknowledgements (ACK messages) in `timeReply` channels (Fig. 2). However, a process may not want to receive signal from time server when waiting for input. Thus, there is a `timeRegistration` channel where client processes can send subscribe or unsubscribe signals to (SUB/UNSUB). At the beginning of every cycle, the `ClockTicking` server updates the subscribed process list. And only the processes in that list will be notified.

```
#define N_SUBJECTS 20  // Maximum number of processes
#define TIME_LIMIT 480  // 8 hours * 60 minutes
mtype:messageType = { SUB, UNSUB, TICK, ACK };
chan timeRegistration = [N_SUBJECTS] of { mtype:messageType, byte };  // SUB/UNSUB
chan timeNotify[N_SUBJECTS] = [1] of { mtype:messageType };  // TICK channels.
chan timeReply[N_SUBJECTS] = [0] of { mtype:messageType };  // ACK channels.
int globalTime = 0;
bool isClosed = false;
active proctype ClockTicking() {
    bool isSubscribed[N_SUBJECTS];  // Mark current subscribed processes.
    do
        :: {
            do  // Check ALL pending registrations (SUB/UNSUB).
                :: nempty(timeRegistration) -> {
                    timeRegistration ? reqMsg, reqId;
                    if
                        :: reqMsg == SUB -> isSubscribed[reqId] = true;
                        :: reqMsg == UNSUB -> isSubscribed[reqId] = false;
                    fi
                }
                :: empty(timeRegistration) -> break;
            od
            for (i : 0 .. N_SUBJECTS-1) {  // Call subscribed processes.
                if
                    :: isSubscribed[i] -> timeNotify[i] ! TICK;
                    :: else -> skip;
                fi
            }
            for (i : 0 .. N_SUBJECTS-1) {  // Wait for subscribed processes.
                if
                    :: isSubscribed[i] -> timeReply[i] ? ACK;
                    :: else -> skip;
                fi
            }
            globalTime++;  // Increase time.
            if
                :: globalTime == TIME_LIMIT -> isClosed = true;
                :: else -> skip;
            fi
            if  // Working overtime: treatment takes longer than expected.
                :: isClosed && _nr_pr == 1 -> {  // Stop when there is no client.
                    break;
                }
                :: else -> skip;
            fi
        }
    od
}
```

Fig. 2. Time server implementation

A typical client process contains two main parts: waiting for input and doing their task. Firstly, the process will be blocked within an alternation in advance of getting input (i.e. a customer) or there is no work left (Fig. 3). The atomic statement is used in order to check the non-emptiness of `inputQueue` and receive the customer from `inputQueue` at the same time, avoiding racing conditions in case of two `SampleDoctors` trying to grab the same customer out of the `inputQueue` channel. This blocking session is run independently of the time server. After obtaining a customer, the process will register to the time server and countdown the working time after every received TICK signal. Each client process in the program is distinguished from each other using process number identification (`_pid`). Once the task is done, which indicates by `workingTime==0`, the process unregisters from the time server and repeat the whole cycle again.

```
chan inputQueue = [10] of { int };  // id of customer
active[2] proctype SampleDoctor() {
  do
  :: {
      atomic {  // Using atomic to check (nempty & get customer) at the same time.
        if // Waiting for input: blocking until getting a customer or going home.
        :: nempty(inputQueue) -> {
            inputQueue?customerId;
          }
        :: isClosed && empty(inputQueue) -> break;
        fi
      }
      select(workingTime : 3..5);  // Randomize working time.
      timeRegistration!SUB,_pid;  // Using _pid to subscribe to time server.
      do
      :: timeNotify[_pid]?TICK -> {  // Receiving TICK signal.
          workingTime--;
          if
          :: workingTime == 0 -> {  // Done working.
              timeRegistration!UNSUB,_pid;  // Unsubscribe from time server.
              timeReply[_pid]!ACK;  // Send last ACK signal.
              break;
            }
          :: else -> timeReply[_pid]!ACK;  // Send ACK signal.
          fi
        }
      od
    }
  od
}
```

Fig. 3. Sample time client

The server-client model for time synchronization is heavily inspired by the Observer pattern, otherwise known as publish-subscribe pattern, described by E. Gamma et al. [1]. The Subject in this project is the time server while the Observers are the other processes in the system (e.g. Doctors). Registered processes are maintained by the time server and upon time updating, the time server informs those registered processes. The ability to subscribe and unsubscribe to the time server allows client processes to stay in blocked status if needed, while keeping the entire program synchronized, sharing the same timing system.

2.2 Customer Entrance Queues (Nguyen Kim Minh Thanh – 18735)

The system makes use of a stochastic generator defined in CustomerEntranceQueue to simulate patient arrivals. Incoming patients are divided into three groups by this procedure: VIP, Insurance, and Normal (in figure 4). The generation logic closely adheres to a probability model in which 40% of the time are normal consumers, 20% are insurance customers, and 10% are VIPs. The remaining 30% are inactive periods with no new arrivals. Each creature is randomly assigned to one of the three hospital departments after being created and given a globally unique ID. The admission workflow is then started by logging and pushing the produced customer object into the entrance channel.

Concurrency: Noted the active state with three parallel instances operational.

Probability: Covered the 40% NORM, 20% INS, 10% VIP, and 30% Skip distribution.

Safety: Referenced the atomic block for identification creation (mitigates mistakes).

Routing: Indicated the arbitrary allocation to Department A, B, or C.

Indicated the transmission of data to the customer entrance channel.

```
active[3] proctype CustomerEntranceQueue() {
   Customer newCustomer;
    bool isSkip;
    do
        :: isClosed -> break;
        :: !isClosed -> {
            isSkip = false;
            // Randomly select customer's type.
            if
                :: 1 -> newCustomer.type = NORM;
```

```
                :: 2 -> newCustomer.type = NORM;
                :: 3 -> newCustomer.type = NORM;
                :: 4 -> newCustomer.type = NORM;
                :: 5 -> newCustomer.type = INS;
                :: 6 -> newCustomer.type = INS;
                :: 7 -> newCustomer.type = VIP;
                :: 8 -> isSkip = true;
                :: 9 -> isSkip = true;
                :: 10 -> isSkip = true;
            fi
            if
                :: !isSkip -> {
                    // Get new customer id.
                    atomic {
                        if
                            :: customerUniversalId >= N_CUSTOMER_MAX -> {
                                break;
                            }
                            :: else -> skip;
                        fi
                        newCustomer.id = customerUniversalId;
                        customerUniversalId++;
                    }
                    // Randomly select customer's department.
                    if
                        :: 1 -> newCustomer.dept = A;
                        :: 2 -> newCustomer.dept = B;
                        :: 3 -> newCustomer.dept = C;
                    Fi
                   // Print into terminal
                    mtype:customerType temp = newCustomer.type;
                    printf("\nCustomer Entrance Queue %d: created a customer with
id %d, type %e\n", _pid, newCustomer.id, temp);
                    // Add new customer to the queue
                    customerEntrance ! newCustomer;
                }
                :: else -> skip;
            fi
        }
    od
}
```

Figure 4. Customer Queue

2.3 Gate Keeper (Nguyen Kim Minh Thanh – 18735)

1. Customer Reception and Input Handling: The GateKeeper process functions as the central admission control unit for the hospital system. It retrieves incoming patient entities from the customerEntrance channel while running in a continuous loop. The process keeps an eye on the global isClosed flag to make sure the simulation ends gracefully; if the hospital is closed and there are no

more patients in the admission queue, the GateKeeper process ends execution. (in figure 5)

2. The model imposes a processing period prior to applying admission logic to replicate actual administrative delays (such registration and triage). When a customer is received, the process chooses a random delay of one to five minutes. To make sure the delay is appropriately reflected in the global simulation time, the GateKeeper uses the global clock synchronization protocol during this time, sending SUB requests and exchanging TICK/ACK signals.

3. Time-Based Admission Logic and Capacity: Whether a department can take on a new patient without going over its daily operating limits is decided by the key decision-making logic. GlobalTime <= TIME_LIMIT - (Current_Queue + 1) * Average_Treatment_Time (in figure 5) is the algorithm the system uses to determine the projected completion time for the current backlog. To avoid additional backlog formation, the patient is denied and the relevant department is marked as closed if the predicted completion time surpasses the simulated limit.

4. Department A's Priority Resource: Reservation Department A has a unique "Phantom Reservation" system for VIP clients, whereas Departments B and C adhere to conventional admissions procedures. The system instantly increases the reservedMachines_DeptA counter inside an atomic block when a VIP is approved by the GateKeeper (in figure 5). In order to prevent lower-priority patients from using high-priority resources while the VIP is in transit, this logic preemptively secures a treatment machine before the VIP physically arrives at the department.

5. Handling Rejections and Routing: The system sends the customer entity to the customerHallway channel (in figure 5) for transit after a successful admittance, increasing the waiting counter for the target department. On the other hand, the system records the rejection event and modifies the system state to indicate that the particular department is now closed to new arrivals if a customer is sent away because of time limits.

```
active proctype GateKeeper() {
    byte processingTime;
    Customer processingCustomer;
    do
        :: {
            // Non-critical: Wait for the next customer.
            if
```

```
                :: customerEntrance ? processingCustomer -> skip;
                :: isClosed && empty(customerEntrance) -> break;
            fi

            // Checking customer takes randomly 1-5 minutes.
            select (processingTime: 1 .. 5);

            // Register to the ClockTicking.
            timeRegistration ! SUB, _pid;


            // Critical: checking customer's type & department.
            // (sync with Global Time)
            do
                :: timeNotify[_pid] ? TICK -> {
                    processingTime--;

                    if
                        :: processingTime == 0 -> {
                            // Unregister to the ClockTicking.
                            timeRegistration ! UNSUB, _pid;
                            timeReply[_pid] ! ACK;
                            assert(nWaitingCustomer_DeptB + nWaitingCustomer_DeptC
+ nWaitingCustomer_DeptA <= N_CUSTOMER_MAX);

                            // Done checking => proceed to decide accept/reject.

// --- VIP RESERVATION LOGIC ---
// VIPs reserve a machine the moment they enter (are accepted by GateKeeper).
        if
       :: processingCustomer.type == VIP -> {
             atomic { reservedMachines_DeptA++; }
    printf("\nA - VIP %d Reserved a machine remotely at %d. Reserved Count: %d\n",
processingCustomer.id, globalTime, reservedMachines_DeptA); }
       :: else -> skip;
          fi
    mtype:customerType temp = processingCustomer.type;
    printf("\nGateKeeper - A(n) %e customer with id %d, going to Department A, is
ACCEPTED at %d\n", temp, processingCustomer.id, globalTime);

      customerHallway ! processingCustomer;
      nWaitingCustomer_DeptA++; }
// The same in another department
```

Figure 5. Gatekeeper

2.4 Hallway (Nguyen Kim Minh Thanh – 18735)

1. Simulation of Hallway Transit: The HallWay technique simulates how patients physically go from the reception gate to their departments. By serving as a dynamic buffer that holds patients for a set period of time, this method mimics walking time. The procedure first enters the admission phase each time the

global clock ticks. During this phase, all pending customers from the customerHallway channel are admitted, stored in a local array, and assigned a random transit duration of one to five minutes.

2. Time Update and Routing Logic: The method decrements the walkers' remaining transit time by iterating through the list of active walkers in the next update phase. A patient is considered to have reached their goal when their timer approaches zero. In order to place the patient in the appropriate particular waiting queue (such as deptQueue_A_VIP or deptQueue_B_Senior), the system next runs a routeing logic block that examines the patient's allocated department (A, B, or C) and priority status (VIP, Insurance, or Normal).

3. Dynamic Memory Management: The procedure uses a shift-left method to effectively manage the list of active walkers inside a static array structure. A patient is eliminated from the array once their transit is over, and they are sent to a queue. In order to ensure that the array stays contiguous and that the index variable appropriately reflects the current number of patients in the corridor, the system then adjusts all succeeding entries down by one index to fill the gap.

4. Process Termination: A clean termination condition is part of the procedure. It keeps an eye on both the local index count and the global isClosed flag. In order to ensure that no patients are left stuck in transit at the conclusion of the simulation, the process will only unsubscribe from the clock service and stop running when the hospital is formally closed, and the corridor is entirely empty.

2.5 Department A (Nguyen Huu Trien - 17813)

The implementation of Department A models a resource-constrained environment where the workforce exceeds the available equipment. The department consists of **three doctors** (`active[3]`) but only **two treatment machines** (`freeMachines_DeptA = 2`). This configuration forces the doctors to compete for resources, creating a concurrency challenge that requires careful synchronization to prevent deadlocks or race conditions.

Resource Management and Phantom Reservation As established in the GateKeeper phase, when a VIP customer enters the hospital, a machine is essentially "reserved" for them remotely before they physically arrive at the department. To ensure fairness and priority, the doctors use an **atomic block, shown in Fig. 6,** to check the status of the queues and the machines simultaneously. This ensures that a lower-priority customer (Normal or Insurance) cannot seize a machine that is technically "free" but has been reserved for an incoming VIP.

Code Implementation The critical decision-making logic is encapsulated in **Fig. 6**. This implementation ensures that the check for available patients and the acquisition of the machine happen as a single, indivisible step:

```
atomic {
    if
        // 1. VIP (Highest Priority)
        // Can take a machine if any are physically free.
        :: nempty(deptQueue_A_VIP) && freeMachines_DeptA > 0 -> {
            deptQueue_A_VIP ? customerId;
            freeMachines_DeptA--;
            reservedMachines_DeptA--; // The "Remote Reservation" becomes "Physical Usage"
            hasMachine = true;
            isVIPJob = true;
            currentCustomerType = VIP;
        }
        // 2. Insurance (Middle Priority)
        // Can ONLY take a machine if free count > reserved count.
        :: empty(deptQueue_A_VIP) && nempty(deptQueue_A_INS) && freeMachines_DeptA >
reservedMachines_DeptA -> {
            deptQueue_A_INS ? customerId;
            freeMachines_DeptA--;
            hasMachine = true;
            isVIPJob = false;
            currentCustomerType = INS;
        }
        // 3. Normal (Lowest Priority)
        // Same logic as Insurance.
        :: empty(deptQueue_A_VIP) && empty(deptQueue_A_INS) && nempty(deptQueue_A_NORM) &&
freeMachines_DeptA > reservedMachines_DeptA -> {
            deptQueue_A_NORM ? customerId;
            freeMachines_DeptA--;
            hasMachine = true;
            isVIPJob = false;
            currentCustomerType = NORM;
        }
        // Exit conditions
        :: isClosed && empty(deptQueue_A_VIP) && empty(deptQueue_A_INS) &&
empty(deptQueue_A_NORM) -> break;
    fi
}
```

Fig (6) Atomic Resource Acquisition with Phantom Reservation (Department A)

Explanation of Logic

1. **VIP Handling:** The VIP condition **in Fig. 6** (:: nempty(deptQueue_A_VIP)) checks only if freeMachines_DeptA > 0. Since the VIP already incremented the reservation counter upon entering the hospital, they are entitled to consume that reservation. When a doctor picks up a VIP, they decrement both the

freeMachines_DeptA and the reservedMachines_DeptA counters as seen in the code.

2. **Insurance and Normal Handling:** For non-VIP patients, the condition **in Fig.** is stricter. The doctor can only acquire a machine if freeMachines_DeptA > reservedMachines_DeptA. This ensures that if there are 2 machines free, but 2 VIPs are currently walking down the hallway (reservation count = 2), the doctors will wait and will not treat the Normal/Insurance patients, preserving the resources for the incoming VIPs.

Process Execution Once a machine is acquired (`hasMachine = true`), the doctor process enters the treatment phase. This utilizes the global clock synchronization mechanism to simulate a treatment duration of 10 to 20 minutes. Upon completion, the doctor releases the machine back to the pool (`freeMachines_DeptA++`) inside another atomic block, allowing other waiting doctors to proceed.

2.6 Department B (Nguyen Huu Trien - 17813)

Department B models a hierarchical treatment structure often found in specialized medical wards. Unlike Department A, where all doctors are equal, Department B divides its workforce into **two Junior Doctors** and **one Senior Doctor**. This structure requires a specific "Referral System" logic to handle patient severity and task delegation effectively.

Hierarchical Workflow and Triage The workflow in Department B is designed as a two-stage pipeline for non-VIP patients (Normal and Insurance), while VIP patients bypass the initial stage entirely.

Junior Doctors (First Line of Defense): The two Junior Doctors serve as the initial contact point for Normal and Insurance customers. They operate on a strict priority basis, always selecting Insurance customers from the deptQueue_B_Junior_INS queue before attending to Normal customers in deptQueue_B_Junior_NORM. Upon selecting a patient, the Junior Doctor performs an **Examination Phase** lasting between 3 to 5 minutes to assess the patient's condition.

The Probabilistic Referral Decision: A key feature of Department B is the simulation of medical uncertainty. After the examination, the system executes a randomized decision logic (50/50 probability), as shown in **Fig. 7**, to determine the severity of the patient's condition:

- **Mild Case:** The Junior Doctor proceeds to treat the patient immediately (10-15 minutes).

- **Severe Case (Referral):** The Junior Doctor determines they cannot handle the case. The patient is **not** discharged but is instead transferred internally to the deptQueue_B_Senior.

```
// 2. Decision Phase (Mild vs Severe)
if
    :: 1 -> isSevere = false; // Mild
    :: 1 -> isSevere = true;  // Severe
fi

if
    :: isSevere -> {
        // Refer to Senior (isReferral = true)
        deptQueue_B_Senior ! customerId, currentCustomerType;
        // Note: Patient is still in Dept B, so we DO NOT decrement nWaitingCustomer_DeptB
yet.
    }
    :: !isSevere -> {
        // Treat Mild Case (10-15 minutes)
        // ... (treatment logic) ...
    }
fi
```

Fig (7) Probabilistic Referral Logic (Department B)

Senior Doctor (Specialist Handling): The Senior Doctor operates a single distinct process that manages the most critical cases. Their workload comes from VIPs (entering directly) and Referrals (passed up by Junior Doctors). To reflect the difficulty of the tasks, the Senior Doctor's treatment times vary based on the patient type .

2.7 Department C (Nguyen Huu Trien - 17813)

Department C represents the most resource-intensive section of the hospital, modeled as a synchronous three-stage pipeline: **Pre-OP Room**, **Operating Rooms**, and **Cleaning Team**. This structure introduces strict dependencies between stages—an Operating Room cannot accept a patient until the Pre-OP room is finished, and it cannot function again until the Cleaning Team has sanitized it.

The Pipeline Stages

- **Stage 1: The Pre-OP Room (The Holding Buffer):** This single resource acts as a bottleneck. Patients must stay for 3-5 minutes to prepare for surgery.
- **Stage 2: The Operating Rooms (The Critical Section):** There are **two Operating Rooms**. A surgeon must atomically verify that a specific room is CLEAN and the isPreOPReady flag is true before pulling a patient.
- **Stage 3: The Cleaning Team:** Once surgery is complete, the room becomes DIRTY. The Cleaning Team occupies it for 5-10 minutes before resetting it to CLEAN.

The "Preemption" Logic (VIP Kicking) The most unique concurrency solution in Department C is the **Preemption Mechanism**. Because there is only one Pre-OP room, a VIP arriving while the room is occupied by a standard patient causes a conflict.

To solve this, the system implements a "Kick-out" rule illustrated in **Fig. 8**. The Pre-OP process constantly monitors the deptVIPQueue_C. If a VIP arrives while a Normal or Insurance patient is being prepped, the system:

1. **Ejects** the current patient back to the general waiting queue (deptQueue_C).
2. **Replaces** them immediately with the VIP.
3. **Restarts** the preparation timer for the VIP.

```
// Check if the current customer is kicked out or not.
if
    :: preOPCustomerType != VIP -> {
        if
            :: nempty(deptVIPQueue_C) -> {
                byte vipId;
                deptVIPQueue_C ? vipId

                // The current customer is kicked out of the Pre-OP room.
                deptQueue_C ! preOPCustomerID, preOPCustomerType;  // Requeue the current
customer.

                // The next customer is a VIP.
                isPreselected = true;
                preOPCustomerID = vipId;
                preOPCustomerType = VIP;

                // End the countdown.
                timeRegistration ! UNSUB, _pid;
                timeReply[_pid] ! ACK;
                break;
            }
            :: empty(deptVIPQueue_C) -> skip; // There is no VIP atm.
        fi
    }
    :: else -> skip; // VIP cannot be kicked.
fi
```

Fig. (8). Preemption Logic "Kick-out" Rule (Department C)

This aggressive logic ensures that VIPs effectively have "zero wait time" for the Pre-OP resource, even if it requires undoing the progress of a lower-priority customer .

Conclusion (Nguyen Huu Trien - 17813)

This project modeled a resource-constrained hospital to address concurrent allocation challenges, demonstrating that strict priority rules can be maintained without sacrificing stability . The implementation highlights three distinct synchronization patterns used to solve specific concurrency problems:

Atomic Resource Locking (Department A), shown in Fig. 6, proved that "Phantom Reservations" can be managed safely. It verified that atomic checks are essential to prevent lower-priority processes from consuming resources logically reserved for incoming VIPs .

Hierarchical Triage (Department B), illustrated in Fig. 7, solved the "Skill-Gap" problem. By using a dynamic referral mechanism rather than a single shared queue, the system effectively balances the load between Junior and Senior doctors.

Preemptive Scheduling (Department C), detailed in Fig. 8, addressed the surgical bottleneck. The implementation confirmed that ensuring zero-wait policies for VIPs requires aggressive preemption—specifically, the "Kick-out" rule to eject lower-priority tasks from the Pre-OP room .

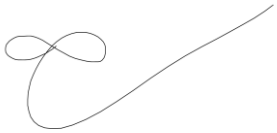
Ultimately, this project underscores the necessity of formal verification tools like SPIN. While the current model relies on a static workforce, it serves as a proof-of-concept that rigid concurrency controls, when applied correctly, guarantee system reliability in the face of unpredictable demand .

Declaration

I hereby declare that my marked part of the report and the Promela code was independently composed/authored by myself. I have not made use of the unauthorised assistance of third parties. Furthermore, I have used only the stated sources or aids and I have referenced all statements (particularly quotations) that I have adopted from the sources I have used verbatim or in essence.

I also declare that this report and the Promela code has not yet been part of another examination process.

Nguyen Phuoc Thanh – 10422075 

Nguyen Huu Trien – 17813 

Nguyen Kim Minh Thanh – 18735 

References

[1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns, Reading: Addison-Wesley, 1994.